CNN MODEL FOUNDATIONS

SmallCNN1D Model Foundations

Architecture, tensor trace, costs, and evidence boundaries

Object	Frozen K=12 PyTorch reference instantiated from the shared `SmallCNN1D` implementation.
Input	Six sensor channels by 128 samples; a 2.56-second window at 50 Hz.
Output	Twelve real-valued logits in the fixed class order recorded in the run configuration.
Checkpoint	Seed 43, best validation macro-F1 at epoch 8; SHA-256 `8ffc5dd8eb0e9cdf832160d074f3ae816ae6cba02adf7214a63a8b623303adb1`.
Evidence boundary	Describes the selected reference function and its observed pointwise behavior. It does not establish regional robustness, a formal certificate, or net ECDR savings.

This companion follows the executable model and the Week 1 locked reference record. It replaces the prior English file's theoretical valid-convolution/GAP model as the description of the actual HAPT reference. That GAP construction remains a separate mathematical branch; its identities and counts cannot be transferred to this model.

Contents

1 Scope, evidence, and model identity **2**
1.1 Independent model-study objective and questions . . . 2
1.2 Method and evidence boundary . . . 2
1.3 Why a one-dimensional CNN for HAPT windows . . . 3

2 Input semantics and preprocessing **3**

3 Layer-by-layer tensor trace **4**
3.1 Convolution and bias . . . 5
3.2 Batch normalization and inference folding . . . 5
3.3 ReLU, max-pooling, and flatten . . . 5

4 Parameter ledger **6**

5 Compute and activation cost **7**

6 Receptive field and finite boundaries **7**

7 Why global-average-pooling cancellation does not apply **8**

8 Evaluation-mode determinism and numerical equivalence **8**

9 Training provenance and selected reference **9**
9.1 Per-class supervised test results . . . 10

10 Decision margins: point values versus regional guarantees **12**

11 Connection to safe event-conditioned reuse **12**

12 Latency, memory, and energy boundaries **13**

13 Limits and interpretation **13**

14 Independent study conclusion **14**

15 Evidence map and reproduction pointers **15**

16 Handoff **15**

1 Scope, evidence, and model identity

The object analyzed here is the exact K=12 reference selected by the Week 1 pipeline. The implementation is `scripts/backbone.py`, class `SmallCNN1D`; the controlling data and architecture record is `00_OVERVIEW/week1/SOURCE_OF_TRUTH.md`. The selected checkpoint and its matching preprocessing parameters live under `output/week1_closeout_20261102/reference/`. Configuration says input shape $(6, 128)$, channels $32, 64, 64$, kernels $5, 3, 3$, two factor-two max-pools, hidden width 128, ReLU and BatchNorm. The final layer has 12 outputs. The model contains 283,692 trainable scalar parameters.

The replaced pre-Week-2 English source has been preserved at `output/week2_foundations_20261103/before/02_MODEL/01_ECDR_CNN_Model_Analysis_EN.tex`. It remains an archive of the earlier theoretical exposition, but its valid-convolution/GAP network is not a description of the frozen HAPT reference. The Vietnamese companion was not part of this replacement.

The report distinguishes three claims. First, source facts: module order, shapes, padding arguments, and tensor operations read from the source. Second, artifact facts: checkpoint identity, stored logits, metrics and timing produced by the run and independent audit. Third, mathematical consequences: parameter formulas, receptive field, and sufficient conditions for a regional class guarantee. Each consequence states its assumptions. In particular, pointwise test accuracy is neither robustness nor the ECDR certificate predicate.

The reference identity includes both learned weights and the train-only channel normalization. The preprocessing record has ID `aa8e1cc20829307187e0a78200f9957f7dd068b1dc26063c2720ad832bb2fd6e`. Changing either weights or normalization changes the classifier function and invalidates claims tied to this frozen identity.

1.1 Independent model-study objective and questions

This report is an independent Week 2 study of the frozen model as an object of measurement and later formal analysis. It asks: (RQ1) what exact function is instantiated, including tensor dimensions, normalization, learned and non-learned state; (RQ2) what structural costs and temporal dependencies follow from that graph; (RQ3) what does the selected checkpoint achieve on the declared supervised test subset, by class and in its full-stream pointwise margins; and (RQ4) which facts are sufficient for a pointwise reproduction, and which additional obligations remain for certified decision reuse? The questions concern this one selected reference, not architecture search or a comparison of candidate networks.

1.2 Method and evidence boundary

The method is a frozen-artifact audit. Source inspection traces `SmallCNN1D.__init__` and `forward` in `scripts/backbone.py`; artifact inspection binds the selected weights, class order, run configuration, preprocessing arrays and stored predictions to their hashes. A separate inference-only reconstruction compares fresh outputs against stored outputs over all 5,053 chronological windows. Label-based metrics use the 3,572 rows admitted by the evaluation's supervised-label rule; unlabeled/excluded rows remain in the full-stream prediction comparison but not the confusion matrix. Structural counts are independently derived from the instantiated modules and formulas in this report. The supplied audit ledger records 18 named checks, including exact preprocessing, row identity, prediction identity, metric agreement, parameter/MAC/receptive-field counts, and evaluation-mode repeatability. No training, hyperparameter selection, alternative architecture experiment, regional verifier, or isolated performance experiment is part of this method. Thus findings are ver-

ified for the bound artifacts and setup, while broader claims about generalization or certification remain unestablished.

1.3 Why a one-dimensional CNN for HAPT windows

The input is an ordered sequence of six row-aligned sensor channels; physical clock synchronization was not independently measured. A temporal Conv1d applies learned filters to local neighborhoods along sample index while combining channels; its shared kernel is reused at successive temporal positions. This matches two useful structural properties of the representation: nearby samples can jointly encode short motion patterns, and the same kind of local pattern may occur at more than one position within a window. The first convolution spans five samples (0.10 seconds at 50 Hz); after the two pooling stages, the final convolution's nominal field is 20 original samples (0.40 seconds), with adjacent final positions four samples apart. Hierarchical filters can combine lower-level local responses into broader temporal features.

These are architectural motivations, not empirically isolated benefits in this study. The six axes are jointly processed by the first convolution, but there is no explicit sensor-physics constraint or channel-invariant mechanism. Weight sharing reduces the need to learn unrelated local filters at every location, while zero padding, fixed pool phase and the flatten-plus-dense head leave boundary and absolute-position sensitivity. The dense head can assign separate coefficients to each of 2,048 channel-position coordinates. Therefore this selected network is not globally translation invariant or guaranteed to recognize an activity independent of position. Its symmetric local kernels are not per-sample causal filters: a feature at an interior sample can use later samples within the window. Evaluating a completed window at its endpoint can still use only measurements already acquired; these two timing interpretations must be distinguished. No ablation against a recurrent model, transformer, MLP, GAP model, or alternative kernel width was run; this choice is frozen provenance, not a demonstrated accuracy-optimal design.

2 Input semantics and preprocessing

Let $X \in \mathbb{R}^{B\times 6\times 128}$ denote a batch, where B is batch size, channel $c \in \{0,\dots,5\}$ indexes the six accelerometer and gyroscope signal axes, and sample index $t \in \{0,\dots,127\}$ is time. At 50 Hz the sample interval is $1/50 = 0.02$ seconds; 128 samples span $128/50 = 2.56$ seconds under the dataset's window convention. The model is not passed a two-dimensional image: the last coordinate is temporal and convolution mixes channel values over local time neighborhoods.

For each channel, the pipeline estimates a mean and standard deviation using training windows only, pooling over the training-window and time axes. Denote these fixed values by μ_c and s_c, with $s_c \geq 10^{-6}$. For every split, including validation and test, the transformed value is

$$Z_{b,c,t} = \frac{X_{b,c,t} - \mu_c}{s_c}.$$

The same six pairs are reused at inference. This is a channelwise affine change of coordinates; it is not normalization recomputed per window. Computing test-set statistics at run time would define a different function and would leak information across the declared evaluation boundary.

The HAPT window builder retains the complete chronological stream, including ambiguous and unlabeled windows. Supervised training and label-based evaluation select only

accepted pure or majority windows; full-stream inference computes outputs for every retained window. The CNN forward receives only the numeric tensor. Labels are supplied to the loss or evaluator, not as network inputs. It does not receive subject identifiers, event descriptors, raw timestamps, or a causal stream state. The model therefore maps one isolated normalized window to logits. Any event logic, temporal candidate selection, region membership, cache state, or fallback belongs to a system layer outside this network.

As an arithmetic illustration, if one channel has training mean $\mu_c = 2$ and clipped standard deviation $s_c = 0.5$, a raw sample of 2.5 becomes $(2.5 - 2)/0.5 = 1$. The model uses its stored per-channel constants. A raw perturbation δX maps to normalized perturbation $\delta Z_{c,t} = \delta X_{c,t}/s_c$. Thus a candidate radius in normalized coordinates represents a channel-rescaled region in raw units. A certificate must state which coordinate system defines its radius.

3 Layer-by-layer tensor trace

All shapes below include the batch axis. BN1d denotes BatchNorm1d. The table follows the actual sequential module indices in `SmallCNN1D`; B is arbitrary and the selected reference uses $B = 1$ for latency measurement.

Module	Operation	Input	Output	Local role
conv.0	Conv1d $6 \to 32$, $k = 5, p = 2$	$(B, 6, 128)$	$(B, 32, 128)$	Learn 32 temporal filters; zero padding preserves length.
conv.1	BN1d(32)	$(B, 32, 128)$	$(B, 32, 128)$	Channelwise affine normalization using batch statistics in training and running statistics in evaluation.
conv.2	ReLU	$(B, 32, 128)$	$(B, 32, 128)$	Elementwise positive-part nonlinearity.
conv.3	MaxPool1d(2)	$(B, 32, 128)$	$(B, 32, 64)$	Maximum in each non-overlapping pair; temporal stride 2.
conv.4	Conv1d $32 \to 64$, $k = 3, p = 1$	$(B, 32, 64)$	$(B, 64, 64)$	Mix prior features locally; preserve this stage's length.
conv.5	BN1d(64)	$(B, 64, 64)$	$(B, 64, 64)$	Per-feature affine normalization.
conv.6	ReLU	$(B, 64, 64)$	$(B, 64, 64)$	Elementwise gate.
conv.7	MaxPool1d(2)	$(B, 64, 64)$	$(B, 64, 32)$	Reduce temporal positions by two.
conv.8	Conv1d $64 \to 64$, $k = 3, p = 1$	$(B, 64, 32)$	$(B, 64, 32)$	Final local temporal mixing.
conv.9	BN1d(64)	$(B, 64, 32)$	$(B, 64, 32)$	Evaluation uses fixed running moments.
conv.10	ReLU	$(B, 64, 32)$	$(B, 64, 32)$	Nonnegative final feature tensor.
head.0	Flatten	$(B, 64, 32)$	$(B, 2048)$	Preserve channel-major positional order.
head.1	Linear $2048 \to 128$	$(B, 2048)$	$(B, 128)$	Dense position- and channel-specific combinations.
head.2	ReLU	$(B, 128)$	$(B, 128)$	Hidden activation.
head.3	Linear $128 \to 12$	$(B, 128)$	$(B, 12)$	Emit class logits.

The length-preservation formula for a one-dimensional convolution is $L_{out} = \lfloor (L + 2p - d(k-1) - 1)/s + 1 \rfloor$. Here dilation $d = 1$, stride $s = 1$, and padding $p = (k-1)/2$, so odd kernels 5 and 3 preserve lengths 128, 64 and 32. The two pools use kernel 2, stride 2 and no padding, exactly halving the even lengths. These shape equalities are implementation consequences, not learned properties.

3.1 Convolution and bias

For input feature map $u \in \mathbb{R}^{C_{in} \times L}$, output channel o, temporal location t, kernel width k, padding p, weights $W \in \mathbb{R}^{C_{out} \times C_{in} \times k}$ and bias $b \in \mathbb{R}^{C_{out}}$, convolution is

$$v_{o,t} = b_o + \sum_{i=0}^{C_{in}-1} \sum_{q=0}^{k-1} W_{o,i,q}\, \widetilde{u}_{i,t+q-p},$$

All three instantiated convolutions use stride one, dilation one, groups equal to one, and a bias. Here $\widetilde{u}$ is zero outside the finite sequence. This is cross-correlation in the common deep-learning convention (the kernel is not reversed). A filter is shared across temporal positions, but not across output channels. At boundary positions, some terms multiply padding zeros; the output still exists because the explicit padding is part of the operator.

The first convolution can combine all six sensor axes over five neighboring samples. The second combines 32 learned features across three positions, and the third combines 64 features across three positions. A local filter may learn useful patterns, but the architecture alone says nothing about whether a particular filter corresponds to a physical activity or causal mechanism.

3.2 Batch normalization and inference folding

For channel c, BatchNorm transforms activation $v_{b,c,t}$ to

$$y_{b,c,t} = \gamma_c \frac{v_{b,c,t} - \widehat{\mu}_c}{\sqrt{\widehat{\sigma}_c^2 + \epsilon}} + \beta_c.$$

Training estimates moments from the current batch and temporal positions; evaluation uses stored running mean m_c and variance r_c. The module has trainable scale γ_c and offset β_c, and non-trainable running buffers plus a batch counter. The three BatchNorm modules therefore contribute 64+128+128 trainable values, while their buffers are required to reproduce evaluation behavior. Running moments and the counter occur in the state dictionary. The value $\epsilon = 10^{-5}$ is an attribute supplied by the reconstructed module constructor rather than a tensor in that dictionary; reproducing the function therefore requires matching the software/operator contract as well as loading the saved tensors.

When evaluation mode is active, BatchNorm is a fixed affine map. If the immediately preceding convolution has weight $W_{o,i,q}$ and bias b_o, define

$$a_o = \frac{\gamma_o}{\sqrt{r_o + \epsilon}}, \quad W'_{o,i,q} = a_o W_{o,i,q}, \quad b'_o = a_o(b_o - m_o) + \beta_o.$$

Then evaluation-mode Conv followed by BatchNorm is algebraically equivalent in real arithmetic to $\mathrm{Conv}(W', b')$ followed by the same activation. This folds only the evaluation function. It does not describe training, where batch moments vary. Floating-point kernels may round differently, so deployment equivalence must be measured at the relevant tolerance; algebraic equivalence alone does not promise bitwise identical logits.

3.3 ReLU, max-pooling, and flatten

ReLU is $\mathrm{ReLU}(a) = \max(0, a)$ applied elementwise. It is continuous and 1-Lipschitz in common norms, but piecewise affine: its active/inactive pattern can change at zero. A local linear expression is valid only while the activation pattern stays fixed or when a sound bound covers all patterns.

For each channel, max-pooling maps a pair (a_{2j}, a_{2j+1}) to $\max(a_{2j}, a_{2j+1})$. The maximum is continuous and 1-Lipschitz in the infinity norm on that pair. Its selected argmax index can switch at a tie boundary. A certificate method that replaces max-pooling with a fixed linear selector has to prove the selector remains valid throughout the candidate region; evaluating the selector only at the center is insufficient.

Flatten imposes an ordering. With channel index c and final position j, one common contiguous ordering maps (c, j) to coordinate $32c + j$. The dense matrix can attach a different coefficient to every such coordinate. Thus it retains positional information and has no inherent translation invariance. The preceding shared convolutions do not remove this positional sensitivity because pooling and the dense head preserve absolute position in the flattened vector.

4 Parameter ledger

For a Conv1d with bias, parameters are $C_{out}C_{in}k + C_{out}$. BatchNorm contributes two trainable vectors of length C_{out}. A Linear layer contributes $n_{out}n_{in}+n_{out}$. Applying those formulas gives the exact ledger below.

Layer	Calculation	Trainable values
Conv1	$32 \cdot 6 \cdot 5 + 32$	992
BN1	$2 \cdot 32$	64
Conv2	$64 \cdot 32 \cdot 3 + 64$	6,208
BN2	$2 \cdot 64$	128
Conv3	$64 \cdot 64 \cdot 3 + 64$	12,352
BN3	$2 \cdot 64$	128
Linear1	$128 \cdot 2048 + 128$	262,272
Linear2	$12 \cdot 128 + 12$	1,548
Total	sum	283,692

Most trainable values are in the first dense layer: 262,272 / 283,692, approximately 92.45 percent. This is a structural observation, not a claim that the dense layer dominates measured latency or energy. The parameter count excludes BatchNorm buffers and runtime activations. A float32 copy of trainable weights alone occupies $283,692 \times 4 = 1,134,768$ bytes (about 1.08 MiB); a serialized state also contains buffers and metadata, and actual runtime memory depends on backend copies, allocator behavior and resident tensors.

The parameterized class supports alternative class counts. Changing K changes only the final layer's output rows and bias in the model definition, but it also changes checkpoint identity and the semantic class map. Here K=12 is fixed by the frozen reference. The historical K=6 report and artifacts do not supply evidence for this 12-class checkpoint.

The first dense layer is large because flattening yields $64 \times 32 = 2,048$ positional coordinates. Each of 128 hidden units receives a distinct coefficient for each coordinate, giving 262,144 weights before bias. Substituting pooling for flattening would reduce positional coordinates but change the model function; it would require a separately trained and evaluated reference.

5 Compute and activation cost

We count one multiply and its accumulation as one multiply-accumulate (MAC). Bias additions, BatchNorm, ReLU, pooling, memory movement and softmax are excluded, so the total is a layer-arithmetic count rather than a runtime model.

$$
\begin{aligned}
M_{conv1} &= 128(32)(6)(5) = 122{,}880,\\
M_{conv2} &= 64(64)(32)(3) = 393{,}216,\\
M_{conv3} &= 32(64)(64)(3) = 393{,}216,\\
M_{fc1} &= 2048(128) = 262{,}144,\\
M_{fc2} &= 128(12) = 1{,}536.
\end{aligned}
$$

The total is 1,172,992 MACs per window under this convention. Using the reporting convention of two floating-point operations per MAC gives 2,345,984 counted FLOPs, excluding the same other operations. This convention is not an instruction trace or an exact count of fused backend operations. A paper or benchmark must state its convention before comparing totals.

The forward activation tensors have these element counts per sample: input 768; first convolution/BN/ReLU 4,096; first pooled output 2,048; second convolution/BN/ReLU 4,096; second pooled output 2,048; third convolution/BN/ReLU 2,048; hidden vector 128; logits 12. The largest listed intermediate is 4,096 float32 values, or 16 KiB per sample, before accounting for backend workspaces and copies. Training retains additional activations for gradients, and batch size multiplies tensor storage. Parameter memory, activation memory, workspace/allocator peak, model-file size, and energy are distinct quantities.

6 Receptive field and finite boundaries

For a sequence of local operators, let r_ℓ be the nominal number of original input samples covered by one output at layer ℓ, and j_ℓ the spacing, in original samples, between adjacent outputs. With kernel k, stride s, dilation one, the standard recurrence is

$$
r_{\ell+1} = r_\ell + (k-1)j_\ell, \qquad j_{\ell+1} = sj_\ell,
$$

starting at $r_0 = j_0 = 1$. Applying it gives:

Operator	Kernel / stride	r samples	j samples
Conv1	5 / 1	5	1
Pool1	2 / 2	6	2
Conv2	3 / 1	10	2
Pool2	2 / 2	12	4
Conv3	3 / 1	20	4

Thus an interior final feature location has a nominal 20-sample receptive field and adjacent output centers are four original samples apart. At 50 Hz this nominal span is $20/50 = 0.4$ seconds. The 32 final positions arise from striding and length reductions, not from 32 independent raw-time blocks: neighboring receptive fields overlap. The fully connected layer combines all 32 final positions, so each logit can depend on the entire 128-sample window. The 20-sample number is a local feature field, not the classifier's complete input support.

At finite boundaries, zero padding means the nominal field partly references padded values rather than observed samples. The convolution stack is not exactly translation equivariant at the edges: shifting an input changes which content is truncated or mixed with padding. Pooling additionally fixes a phase by grouping positions $(0,1),(2,3),\ldots$ at each pooling stage. A one-sample shift can change pair membership and argmaxes even away from a simple boundary explanation. These effects matter when proposing temporal shift equivalence or proving a perturbation region: the proof must use the implemented padding and pool alignment.

For a fixed-length pooling illustration, activations $(1,4,3,2)$ produce $(4,3)$. A one-sample right shift with zero insertion and terminal truncation gives $(0,1,4,3)$ and pooled output $(1,4)$. This illustrates sensitivity to an arbitrary one-sample shift, not the exact HAPT stride. The HAPT stride 64 is divisible by the accumulated pooling stride four, so it preserves pool phase on corresponding interior coordinates. Finite-window padding and the position-sensitive dense head still prevent a general decision-invariance conclusion.

7 Why global-average-pooling cancellation does not apply

The separate theoretical `ECDRConv1DNet` branch ends with global average pooling (GAP). For a feature map $F \in \mathbb{R}^{C\times T}$, GAP is $g_c = T^{-1}\sum_{t=0}^{T-1} F_{c,t}$. Under suitable same-length circular or boundary assumptions, time shifts can preserve this average, which can support a cancellation identity for an affine classifier on the pooled vector. Those are assumptions of that model and transformation.

The selected SmallCNN1D instead maps $F \in \mathbb{R}^{64\times 32}$ to a 2048-vector and computes $h = \mathrm{ReLU}(A\,\mathrm{vec}(F) + a)$. A logit is then $q_k^\top h + d_k$. Its coefficient on position (c,t) can differ from its coefficient on $(c,t+1)$, and the hidden ReLU can also change activation regions. There is no architectural sum that cancels a position permutation. Consequently, a GAP shift-invariance proof cannot be reused for SmallCNN1D by changing only the parameter count or class count.

The older `ecdr/model.py` object `ECDRConv1DNet` has a different layer sequence, global pooling and approximately 20,332 parameters in the recorded K=12 audit. It is a distinct function. The 283,692 parameter SmallCNN1D is the HAPT frozen reference whose decisions the current ECDR system must preserve. Any future comparison between them would need an explicitly matched experiment; parameter count alone is not an accuracy, safety or latency comparison.

8 Evaluation-mode determinism and numerical equivalence

With frozen weights, frozen normalization buffers, evaluation mode, deterministic input, and fixed software/operator behavior, the reference computes a stateless function from normalized tensor to logits. It stores no prior-window memory in the module. This is the right mathematical object for a pointwise classifier $f : \mathbb{R}^{6\times 128} \to \mathbb{R}^{12}$.

The Week 2 model audit reports evaluation-mode BatchNorm determinism and exact agreement between independently reconstructed stored logits for the selected checkpoint (maximum absolute difference 0.0 in that audit). It covered 5,053 windows, with zero argmax disagreements and exact normalization agreement. This validates that reconstruction path within that setup, not every real-valued input or device/library combination. No candidate-region radius or regional lower bound was computed. For formal reasoning, certify the

real-valued operation graph with a numerical soundness bridge or include bounded implementation error in the decision-margin bound.

If two logits differ componentwise by at most η, a pairwise margin can shift by at most 2η: for classes c, j, $(f_c - f_j)$ changes by at most $|\delta_c| + |\delta_j| \leq 2\eta$. Therefore a computed lower bound must retain enough margin after both region-bound looseness and numerical error. Equality at zero does not establish a unique class; strict positivity is the safe condition.

9 Training provenance and selected reference

The checkpoint was selected from three seeds by validation macro-F1. The recorded configuration specifies 30 maximum epochs, batch size 128, Adam with learning rate 10^{-3}, weight decay 10^{-4}, cosine scheduling to 10^{-5}, patience 10 and no class weights. Seed 43 attained best validation macro-F1 0.846785 at epoch 8 and was selected. Its SHA-256 is shown on the cover and in `selection.json`. This provenance is supplied to identify the frozen model, not as a proposal to retrain it.

The selected seed's supervised test denominator is 3,572 labeled windows from the K=12 test stream. Stored evaluation gives accuracy 0.89054 and macro-F1 0.77626. The full stream contains 5,053 windows; the difference reflects unlabeled or excluded examples in the supervised metric denominator, not missing network output. Ground-truth error count is 391 among the supervised test examples. These are descriptive baseline measures of agreement with labels. They do not state whether a reuse method preserves the reference prediction.

For each supervised example with true class y, the ground-truth margin is $m_y(x) = f_y(x) - \max_{j \neq y} f_j(x)$; it may be negative when the model is wrong. The winning margin is $m_{\hat{y}}(x) = f_{\hat{y}}(x) - \max_{j \neq \hat{y}} f_j(x) \geq 0$, where $\hat{y} = \arg\max_j f_j(x)$. The stored Week 1 evaluation record reports a supervised ground-truth margin median of 7.5746; the fresh Week 2 audit reproduces a full-stream winning-margin median of 5.9782. The ground-truth summary was not independently recomputed by the Week 2 script. These use different labels and denominators and must not be interchanged. Neither distribution is a lower bound over a neighborhood.

The stored Week 1 evaluation record reports descriptive adjacent agreement rates of 97.82 percent for pure stable pairs (2,560/2,617) and 52.27 percent for pure transition pairs (23/44). Those figures summarize neighboring windows, with stride 64 (50 percent overlap), within recordings. They do not prove that all points in an event-conditioned candidate set share a class; nor do they constitute independent subject-level confidence estimates.

The top-two gap across 5,053 points has minimum 0.0001135, fifth percentile 0.411995, median 5.97824 and 95th percentile 11.88764 logit units. The minimum reveals a sampled near-tie, not the minimum over an unsampled region. A large point gap likewise gives no perturbation sensitivity without a bound on logit changes. Candidate-region radius and regional margins are unavailable; no certificate can be inferred from these statistics.

The mathematical class indices in this report are 1–12, matching the original activity IDs. Stored prediction indices are zero-based 0–11; adding one maps back to the activity ID. The fixed class order is required for that conversion. The audit uses NumPy argmax, whose first maximum in this order determines the point decision at an exact tie. A strict positive regional margin deliberately excludes ties.

The forward graph emits raw logits and contains no Softmax module. In real arithmetic, $p_k = \exp(f_k) / \sum_j \exp(f_j)$ has the same ordering as the finite logits because the exponential is strictly increasing and the denominator is common and positive. Softmax is therefore

unnecessary for the argmax decision and does not turn its winning score into a calibrated probability. Finite-precision overflow, rounding and ties require a numerical contract; the real ordering identity is not a regional certificate.

9.1 Per-class supervised test results

Let TP_k, FP_k and FN_k denote true-positive, false-positive and false-negative counts for class k. The executable definitions in `scripts/week2_model_audit.py:metrics` are $P_k = TP_k/(TP_k + FP_k)$, $R_k = TP_k/(TP_k+FN_k)$, and $F1_k = 2P_kR_k/(P_k+R_k)$, using zero for a zero denominator. Macro-F1 is $\frac{1}{12}\sum_{k=1}^{12} F1_k$.

Table 2 gives precision, recall, F1 and support for each class in the fixed class order. Precision and recall use the usual one-versus-rest definitions; support is the number of labeled test examples whose true class is that row. Macro-F1 is the unweighted mean over the twelve class F1 values, whereas accuracy is the fraction correct over all 3,572 labeled examples. The movement classes have larger supports than most transitions; the lower transition-class scores and smaller denominators make aggregate quality particularly sensitive to those examples. This is descriptive evidence for the selected split, not a population estimate.

Table 2: Per-class supervised test metrics (3,572 labeled windows).

Class	Precision	Recall	F1	Support
WALKING	0.9603	0.9567	0.9585	531
WALKING_UPSTAIRS	0.9331	0.9088	0.9208	537
WALKING_DOWNSTAIRS	0.9237	0.9837	0.9528	492
SITTING	0.7997	0.8407	0.8196	546
STANDING	0.8798	0.8090	0.8429	597
LAYING	0.9831	0.9949	0.9890	586
STAND_TO_SIT	0.7619	0.7111	0.7356	45
SIT_TO_STAND	0.7778	0.6563	0.7119	32
SIT_TO_LIE	0.5893	0.6471	0.6168	51
LIE_TO_SIT	0.7714	0.5870	0.6667	46
STAND_TO_LIE	0.5714	0.5294	0.5496	68
LIE_TO_STAND	0.4737	0.6585	0.5510	41

The confusion matrix in Figure 1 uses true class on rows and predicted class on columns, in the same order. It preserves counts, so diagonal values are correct predictions and off-diagonal values expose the specific observed confusions. The figure is generated from the audited prediction and label rows; it does not include the 1,481 full-stream rows excluded from supervised evaluation.

Figure 2 summarizes the fresh full-stream top-two logit gaps. A gap is the largest logit minus the second largest for one input; every value is nonnegative by definition. It is a pointwise score separation, not a probability, correctness guarantee, perturbation radius, or lower bound valid between samples. The 5,053-point population differs from the supervised subset above.

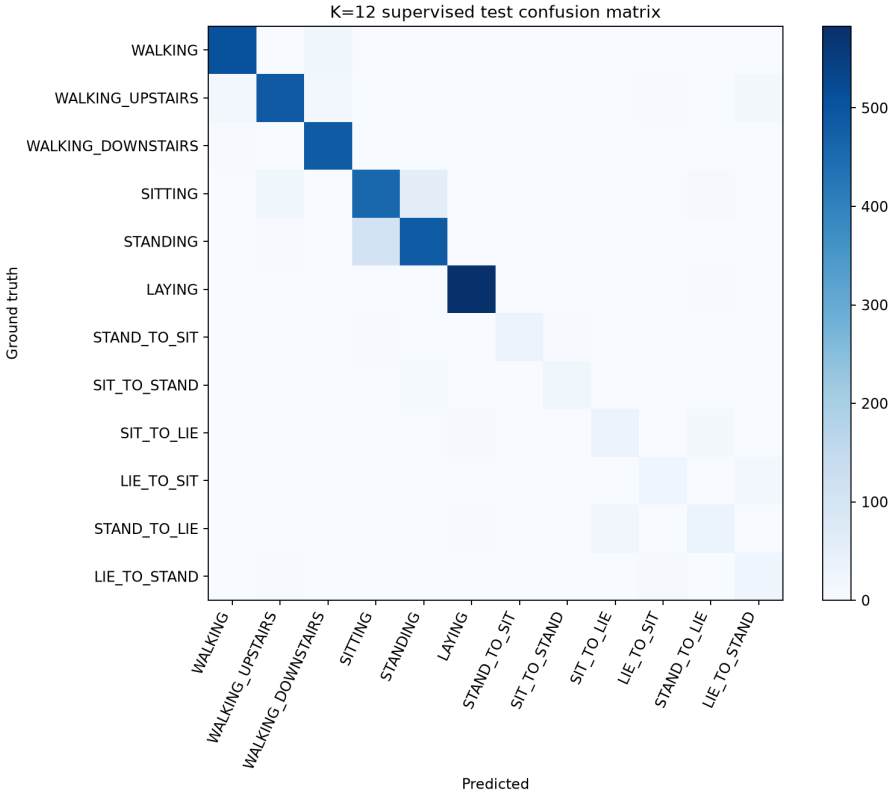


Figure 1: Confusion matrix for the 3,572 supervised test windows. Rows are true labels; columns are predictions.

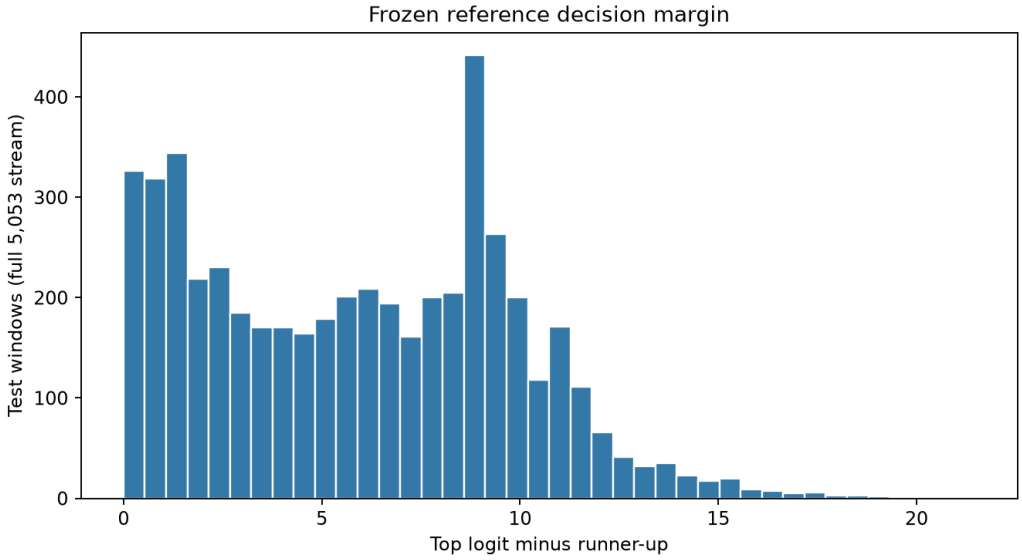


Figure 2: Top-two logit-margin distribution over all 5,053 chronological windows. Pointwise values only.

10 Decision margins: point values versus regional guarantees

The classifier's decision is the largest-logit index. In this section f maps normalized input to logits; the corresponding raw-input map is $F = f \circ N$. Let c be a proposed reference class and let $\mathcal{S}_r \subseteq \mathbb{R}^{6\times 128}$ be a candidate normalized region around normalized anchor X^r. A raw domain must first be mapped using the same fixed N. Pointwise agreement at the anchor says only $f_c(X^r) > f_j(X^r)$ for each competitor j. To guarantee the same unique decision throughout the region, a sufficient condition is

$$\inf_{X\in\mathcal{S}_r} \big(f_c(X) - f_j(X)\big) > 0 \quad \text{for every } j \in \{1,\dots,12\} \setminus \{c\}.$$

There are eleven competing margins. If sound bounds ℓ_j satisfy $f_c(X) - f_j(X) \geq \ell_j$ for every $X \in \mathcal{S}_r$, then $\min_{j\neq c} \ell_j > 0$ proves the label invariant on that region, subject to the numerical contract. A bound for only the strongest competitor at the anchor is insufficient unless the method proves it dominates the other ten competitors throughout the region.

This theorem is about reproducing the classifier's decision. It does not claim that c is the ground-truth activity. If the anchor is misclassified, a valid decision-preservation certificate can preserve the same error. Ground-truth accuracy and reference-decision consistency are separate objectives and need separate denominators and evidence.

The Week 1 margin arrays are point measurements on dataset windows. No such scalar is a substitute for the infimum above. A candidate region may include perturbations, shifts or mixtures that were never sampled. Establishing regional positivity requires a sound bound, verified membership in the declared region, and correct identity of the trusted anchor and classifier. Week 2 reports no successful certificate for the selected model; certification belongs to later work.

11 Connection to safe event-conditioned reuse

ECDR proposes reducing repeated reference evaluations by reusing a previously computed decision when an incoming window belongs to a region for which that decision is certified. A safe reuse path has at least these logical obligations:

1. **Reference identity:** anchor logits and class c are tied to the selected checkpoint hash, its normalization ID, class order, and evaluation contract.
2. **Region definition:** the candidate set $\mathcal{S}_r$ is mathematically specified, with its norm, radius, alignment rules, finite boundary treatment and any event-condition constraints explicit.
3. **Membership:** the current input is proven to belong to that very region. Approximate event similarity is not membership unless the region construction proves the implication.
4. **All-class margin:** a sound procedure bounds all eleven functions $f_c - f_j$ below by a strictly positive value throughout the region, including numerical allowances.
5. **Identity and lifecycle:** cached anchor, region, checkpoint, preprocessing and state-generation IDs match the current execution context. Reset, recording boundary, gap, model update or normalization change invalidates stale state.
6. **Fallback:** if any obligation is unavailable, ambiguous, expired or fails, execute the frozen reference model and emit its output. A failed/timeout verifier cannot silently reuse an old class.

These obligations are system properties. Reference fallback assumes a valid, finite input tensor of the declared shape. Malformed or nonfinite data require explicit rejection/error

handling and cache invalidation; running the network on an invalid input does not establish a trusted anchor. The neural network's architecture alone does not implement these policies. In particular, the model has no event state or fallback pathway; those must be specified and verified by the caller. Week 3 should assign explicit state ownership and reset triggers, then verify that every cache hit and fallback is bound to the same reference identity.

12 Latency, memory, and energy boundaries

The model audit timed batch-one forward calls on a resident pre-normalized tensor, with 100 warmups and 1,000 repetitions, one CPU thread, PyTorch 2.14.1+cpu on an Intel Xeon W-11855M. The final audit completed at 00:21:32 UTC on 3 November 2026 and reports p50 0.155694 ms, p95 0.311076 ms and mean 0.189197 ms. Background CPU workload was not isolated, so these observations are not a controlled performance comparison. Each timed call uses the same first normalized chronological test input. These repeated-input measurements must not be converted into a speed ratio against the Week 1 varied-input measurements. This is an environment-specific forward-only baseline for the selected checkpoint. It excludes preprocessing, window creation, event detection, candidate-region construction, membership checks, verifier work, cache lookup, fallback orchestration, data movement and I/O.

Accordingly, the measurement cannot establish net ECDR savings. A complete comparison must measure matched workloads and include every path: event detection, region building, membership, certificate setup or lookup, fallback frequency, reference forward calls, transfer and I/O. Report wall-clock distributions and the workload composition, alongside reuse coverage and disagreement count. Fewer MACs on a cache hit do not ensure lower end-to-end latency if verification overhead dominates; CPU timing also says nothing about energy on another device.

Memory accounting has similar boundaries. Float32 trainable parameters occupy about 1.08 MiB, while the maximum listed single-sample intermediate is about 16 KiB. A batch multiplies activations; training stores more; implementation workspaces and allocator fragmentation can increase peak resident memory. GPU memory, CPU resident memory, serialized artifact bytes, and energy must be measured separately. No memory or energy reduction is claimed here.

13 Limits and interpretation

The network is a learned discriminative function over finite windows. Its prediction does not by itself provide causal interpretation, calibrated uncertainty, temporal continuity, or proof that an event occurred. ReLU regions and max-pool selectors introduce piecewise-linear structure; this can be useful for analysis, yet exact global enumeration may be expensive. Any relaxation must be sound over the entire candidate region and may be conservative.

The flatten-plus-dense head is position sensitive; its first dense layer dominates parameter count. This increases the importance of preserving flatten order and exact temporal alignment in a reproduction. Pool phase, zero-padding boundaries, normalization constants, BatchNorm buffers, class order, and checkpoint selection are part of the function contract. Replacing any one can yield a valid but different model; results for that model cannot inherit this checkpoint's evidence.

Transition labels are weakly supported in the test split relative to the six basic classes. Aggregate macro-F1 gives every class equal weight but remains sensitive to small class supports. The Week 1 source record states transition-support gates remain failed and overall quality is HOLD. The model remains a feasibility reference for Week 2 analysis, not evidence that the quality gate passed.

14 Independent study conclusion

For RQ1, the source and artifact trace identify a frozen mapping $f : \mathbb{R}^{6\times128} \to \mathbb{R}^{12}$ behind fixed per-channel normalization, with 283,692 trainable values and the checkpoint/preprocessing identities stated above. The pointwise reconstruction is VERIFIED for the audited setup: all 5,053 stored predictions agree with fresh argmax outputs, and fresh logits match the stored artifact within the audit tolerance (reported maximum absolute difference 0.0). The label-based result is accuracy 0.8905375 and macro-F1 0.7762645 on 3,572 supervised rows, with the per-class evidence shown above.

For RQ2, the 1,172,992 MAC count and 20-sample/4-sample-jump local receptive-field result are DERIVED from the traced operators and stated counting/recurrence assumptions. They describe operation structure, not backend instruction counts or empirical speed. The local fields overlap, and the flatten/dense head lets every output depend on all 128 input samples. Weight sharing and local temporal processing explain why this 1D CNN is a plausible modeling choice for ordered sensor windows; this audit does not establish that they improve accuracy over another architecture.

For RQ3 and RQ4, observed class metrics and margins describe only this split and these points. They do not establish generalization, calibrated confidence, event causality, or regional invariance. A future reuse rule must define $F = f \circ N$ on raw inputs, with f acting on the normalized coordinates; bind anchor identity and lifecycle; prove current-input membership in its region; and establish sound strictly positive bounds for all eleven margins $f_c - f_j$ over that region, including a numerical soundness allowance. It must fall back to the frozen reference when any obligation fails and explicitly reject malformed or nonfinite input while invalidating affected state. No regional verifier, membership implementation, complete fallback path, or numerical bridge is established here. Consequently, certified reuse benefit and net end-to-end savings remain NOT ESTABLISHED, and the Week 1 quality status remains HOLD.

The complete review of the earlier 64 concept bodies is recorded in `output/week2_foundations_20261103/concept_audit_64.csv`. It maps every P01–P08 and C01–C56 item to an actual-reference treatment, consolidated background, archived GAP-specific example, or an explicit deferred/unestablished scope. A disposition audit does not imply that every archived training or verifier lesson was re-taught here.

15 Evidence map and reproduction pointers

Claim	Primary evidence	What it supports
Architecture and forward order	`scripts/backbone.py`, `SmallCNN1D.__init__`, `forward`	Source-level layer sequence and shape formulas.
Frozen class/data contract	`00_OVERVIEW/week1/SOURCE_OF_TRUTH.md`	Locked HAPT K=12 semantics and evidence boundary.
Training settings	`output/week1_closeout_20261102/reference/config.json`	Recorded configuration, not a retraining instruction.
Selection identity	`.../selection.json`, selected checkpoint	Seed, epoch, validation criterion and artifact hash.
Metrics and timing	`.../evaluation_results.json` and Week 2 `model_audit.json`	Declared dataset denominators and forward-only timing scope.
Independent structural audit	`output/week2_foundations_20261103/model_audit.json`	Reconstructed shapes, parameters, MACs, RF recurrence and stored-logit comparison.
Reproduction command	`.venv/bin/pythonscripts/week2_model_audit.py`	Rebuilds the model audit JSON under the audit script's stated environment.
Official operator contracts	PyTorch 2.14 documentation: Conv1d, BatchNorm1d, MaxPool1d, Linear	API behavior should be read against the installed version; the local source and audit establish the instantiated arguments.

The reproduction command is evidence collection, not training; it must use the exact checkpoint and normalization artifacts named above. The quantitative tables in this report are tied to the audited files and timestamp. If those artifacts change, the identity and quantitative claims must be rechecked. The human-readable interpretation is `output/week2_foundations_20261103/model_findings.md`; the machine-readable ledger records environment and timing details. The command's completion alone does not certify regional robustness or system-level savings.

16 Handoff

Week 2 fixes the reference function and its measured evidence. Week 3 must supply a sound region bound, conservative membership, all eleven positive class-pair margins, numerical error containment and identity-bound lifecycle/fallback rules. The observed point margins and forward timing do not discharge these obligations.